

Uma comparação entre a técnica de rede neural Perceptron e Árvores de decisão (Python)

Conjunto de dados utilizado: **Wine dataset**

DANILO ALEXANDRE MANUEL

Metas



Escolha do dataset

Analisar os dados e as dimensões

Definir pré-processamento utilizado

Alterações nas configurações dos modelos utilizados

Resultados obtidos

Funcionamento de cada técnica de aprendizado utilizada

Wine Dataset



O conjunto de dados do vinho contém os resultados de uma análise química de vinhos cultivados em uma área específica da Itália. Três tipos de vinho estão representados nas 178 amostras, com os resultados de 13 análises químicas registradas para cada amostra.

class 1, class 2 e class 3.

É um problema de classificação.

Wine Dataset



Os atributos (dimensões) são:

- 1) Alcohol
- 2) Malic acid
- 3) Ash
- 4) Alcalinity of ash
- 5) Magnesium

Wine Dataset



- 6) Total phenols
- 7) Flavanoids
- 8) Nonflavanoid phenols
- 9) Proanthocyanins
- 10) Color intensity
- 11) Hue
- 12) OD280/OD315 of diluted wines
- 13) Proline

Wine Dataset



Classes	3
Divisão de amostras	[59,71,48]
Total de amostras	178
Dimensões	13
Números	reais e positivos

Coletar os dados



Para acessar os dados do dataset eu posso extrair do link

-><http://archive.ics.uci.edu/ml/machine-learning-databases/wine/wine.data>

Ou importar usando `sklearn.datasets import load_wine`

Coletar os dados



Eu preferi usar o `sklearn.datasets` porque já podia dividir o treino e o teste na função `train_test_split()`, ele já preparava tudo.

Usando o link eu podia fazer isso:

```
#Y = data[:, 0]
```

```
#X = data[:, 1:13]
```


Coletar os dados



0	1	2	3	4	5	6	7	8	9	10	11
1	14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04
1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05
1	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03
1	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86
1	13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04
..	...	...	...	...	...	...	...	...	...	...	...
3	13.71	5.65	2.45	20.5	95	1.68	0.61	0.52	1.06	7.70	0.64
3	13.40	3.91	2.48	23.0	102	1.80	0.75	0.43	1.41	7.30	0.70
3	13.27	4.28	2.26	20.0	120	1.59	0.69	0.43	1.35	10.20	0.59
3	13.17	2.59	2.37	20.0	120	1.65	0.68	0.53	1.46	9.30	0.60
3	14.13	4.10	2.74	24.5	96	2.05	0.76	0.56	1.35	9.20	0.61
	12	13									
	3.92	1065									
	3.40	1050									
	3.17	1185									
	3.45	1480									
	2.93	735									
	...	...									
	1.74	740									
	1.56	750									
	1.56	835									
	1.62	840									
	1.60	560									

... (178)

Pré-processamento utilizado (Perceptron)



No perceptron eu usei o pré processamento de normalização MinMax.

A normalização é realizada para garantir que os atributos tenham a mesma ordem de grandeza, o que ajuda o modelo a convergir mais rapidamente e a lidar melhor com os diferentes atributos.

Pré-processamento utilizado (Perceptron)



A normalização dos dados pode ser realizada antes de dividir o conjunto de dados em treinamento e teste, para evitar vazamento de informações do conjunto de teste para o conjunto de treinamento.

Fórmula:

$$X_{\text{norm}} = (X - X_{\text{min}}) / (X_{\text{max}} - X_{\text{min}})$$

Pré-processamento utilizado (Perceptron)



Onde:

X é o valor original do atributo

$X_{\min}$ é o valor mínimo do atributo

$X_{\max}$ é o valor máximo do atributo

X_{norm} é o valor normalizado do atributo após a aplicação da normalização min-max

Pré-processamento utilizado (Perceptron)



Essa técnica é útil quando queremos que os atributos tenham a mesma escala e estejam limitados a um intervalo específico. Além disso, ela preserva a relação de ordem entre os valores originais.

A normalização min-max pode ser aplicada a cada atributo separadamente ou a todo o conjunto de dados. É importante ressaltar que a normalização deve ser aplicada apenas no conjunto de treinamento.

Pré-processamento utilizado (Árvores de decisão)



O pré-processamento realizado inclui a seleção univariada de atributos usando a classe `SelectKBest` do `scikit-learn` com a função de pontuação `f_classif`.

Essa técnica é aplicada aos conjuntos de treinamento e teste para selecionar os atributos mais relevantes antes de treinar o classificador `DecisionTreeClassifier`. Isso ajuda a reduzir a dimensionalidade do conjunto de dados e melhorar a precisão do modelo.

Pré-processamento utilizado (Árvores de decisão)



Na seleção de atributos univariada, a função `f_classif` é aplicada a cada atributo individualmente e os atributos com os maiores valores F (ou menor valor p) são selecionados como os mais discriminantes ou relevantes para o problema de classificação.

Resultados obtidos



Eu procurei calcular a acurácia nos dois e comparar qual chegou no 100% ou mais próximo de 100%.

Perceptron

- Número de iterações necessárias: 14
- Acurácia: 100.0%

Árvore de decisão

Acurácia: 97.2%

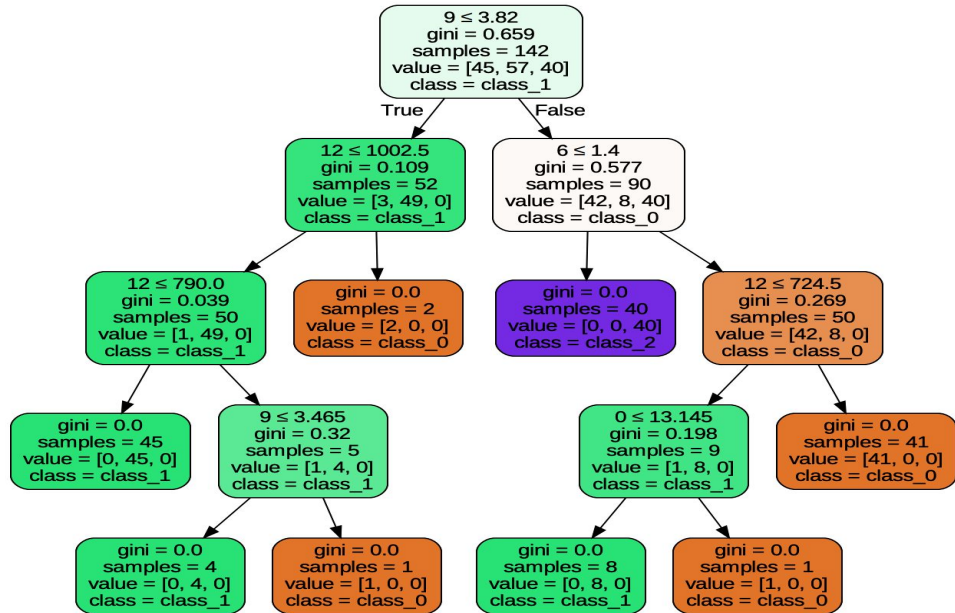
Resultados obtidos



A acurácia do modelo é calculada comparando os rótulos reais `y_test` com as previsões `y_pred` usando a função `accuracy_score()` do módulo `sklearn.metrics`.

Resultados obtidos

Árvore de decisão



Funcionamento de cada técnica de aprendizado utilizada



Perceptron

O Perceptron é um algoritmo de aprendizado de máquina que pode ser usado para classificação binária ou multiclasse.

Ele treina um modelo que consiste em um único neurônio artificial que toma várias entradas ponderadas, aplica uma função de ativação e produz uma saída que representa a classe prevista.

Funcionamento de cada técnica de aprendizado utilizada



Durante o treinamento, os pesos são atualizados iterativamente com base nos erros cometidos pelo modelo, até que uma condição de convergência seja alcançada.

No código o Perceptron é treinado usando o conjunto de treinamento normalizado e usado para fazer previsões no conjunto de teste, em seguida, a acurácia do modelo é calculada para avaliar seu desempenho.

Funcionamento de cada técnica de aprendizado utilizada



Árvore de decisão

A árvore de decisão é um algoritmo de aprendizado de máquina que constrói uma estrutura de árvore hierárquica para tomar decisões com base nas características dos dados de treinamento.

Durante o treinamento, a árvore é construída recursivamente dividindo os dados com base em determinadas regras e critérios de divisão, como a impureza de Gini ou o ganho de informação.

Funcionamento de cada técnica de aprendizado utilizada



Árvore de decisão

Cada nó da árvore representa uma característica e cada ramo representa uma decisão com base nessa característica. No código a árvore de decisão é treinada usando os atributos selecionados pelo SelectKBest e é usada para fazer previsões no conjunto de teste. A acurácia do modelo é calculada para avaliar seu desempenho.



FIM